Contents

NΛCO — Especificação técnica de armazenamento e processamento de imagens **1**
1. Estado e precedência 2
2. Arquitetura e regras de armazenamento 2
    2.1. Componentes e acesso 2
    2.2. Chave, URL e cache 3
3. Entrada e processamento da imagem 3
    3.1. Formatos e limites 3
    3.2. Pipeline obrigatório 4
    3.3. Falha e preservação da imagem anterior 4
4. Persistência e reconciliação de uploads 5
    4.1. `StorageUploadOperation` 5
    4.2. `StorageUploadOperationEvent` 5
    4.3. Tentativas e reconciliação 5
    4.4. Consulta administrativa 6
5. Exclusão e invalidação no Cloudinary 6
    5.1. `StorageDeletionOperation` 6
    5.2. Ordem e semântica da operação 7
    5.3. Falhas e retries 7
6. Concorrência, arquivos temporários e observabilidade . . 8
7. Critérios de aceite e testes mínimos 8
8. Pendências que não bloqueiam a definição conceitual, mas precisam ser fechadas antes da implementação afetada 9
9. Matriz de rastreabilidade Q63–Q103 10
10. Referências oficiais do fornecedor 14

NΛCO — Especificação técnica de armazenamento e processamento de imagens

Versão: 1.2
Data-base: 09/11/2026
Status: decisões Q63–Q103 adaptadas ao Cloudinary; planejamento, não evidência de implementação.
Escopo: imagens de ofertas (`RestaurantProduct`), Cloudinary, operações persistentes e processamento seguro.

> **Identificação das decisões:** Q63–Q103 pertencem à entrevista de decisões sobre imagens e armazenamento. Não são os identificadores `QU-63` a `QU-73` do registro RNF-01. Para evitar colisão, este documento mantém a referência

original Qxx; o registro de desempenho permanece em `03_REGISTRO_DECISOES_RNF01.md`.

1. Estado e precedência

- As decisões Q63–Q103 listadas neste documento foram confirmadas pelo usuário.
- Regras de negócio já aprovadas para catálogo, restaurante e pedidos continuam prevalecendo.
- Este documento detalha a implementação do fluxo de mídia; não declara que código, migrations, conta Cloudinary ou endpoints já existam.
- **Cloudinary foi escolhido para o MVP**, priorizando o plano Free sem cartão de crédito. O Cloudinary concentra armazenamento de mídia, transformações/entrega e CDN; não será adotado bucket privado próprio + CDN independente nesta etapa.
- A aplicação mantém `StoragePort` para isolar o fornecedor e permitir migração futura. O backend continua validando, sanitizando, convertendo e controlando o ciclo de vida das imagens.
- O schema físico, os contratos de upload para o responsável e os valores concretos de timeout/recursos/qualidade ainda precisam ser definidos antes da implementação correspondente. A política de exclusão Q77–Q86 foi adaptada conforme a seção 5; não existe polling de status de purge no contrato padrão do Cloudinary.

2. Arquitetura e regras de armazenamento

2.1. Componentes e acesso

1. Utilizar o Cloudinary como plataforma gerenciada de armazenamento e entrega de imagens, incluindo CDN. Os assets de catálogo são públicos por natureza; não enviar documentos privados ou dados sensíveis para este fluxo.
2. Upload, substituição e exclusão são executados exclusivamente pelo backend. Mobile e web não recebem API Secret nem credenciais administrativas; upload direto do cliente não faz parte do MVP.
3. Manter uma abstração `StoragePort` para desacoplar o domínio do SDK do Cloudinary. O adapter encapsula `upload`, `destroy` com `invalidate=true`, geração de URL e consulta de uso.
4. Usar `public_id` único e imutável por operação, sem nome original do arquivo. Persistir também o `asset_id` imutável e a versão retornada pelo Cloudinary para exclusão e geração de URL versionada.

5. Configurar `cloud_name`, `api_key` e `api_secret` exclusivamente no backend/secret manager. `api_secret` nunca deve chegar ao frontend, logs ou banco em texto aberto.
6. Não habilitar transformações automáticas adicionais do Cloudinary no MVP: o backend entrega o WebP final já processado, evitando créditos extras e diferenças inesperadas de imagem.

2.2. Chave, URL e cache

- O PostgreSQL mantém uma chave lógica interna (`storage_key`) e os identificadores necessários ao adapter: `provider_public_id`, `provider_asset_id` e `provider_version`. Não persistir segredo, token ou URL assinada. O `secure_url` pode ser retornado pela API a partir dos metadados/versionamento do asset, sem assumir que um domínio próprio/CDN separado exista no plano Free.
- Configurar `CLOUDINARY_CLOUD_NAME` por ambiente. Credenciais administrativas ficam em secrets; nunca no app mobile/web.
- Cada upload gera `public_id` único por operação, sem nome original, por exemplo `naco/{environment}/restaurants/{rId}/products/{rpId}/{uploadI`. Não sobrescrever public IDs existentes.
- A versão retornada pelo Cloudinary integra a URL de entrega e funciona como cache-busting. Substituição cria novo public_id/asset e só atualiza a referência no PostgreSQL após upload confirmado e compare-and-swap.
- Não assumir `max-age=31536000, immutable` para a CDN padrão do Cloudinary: assets servidos em URLs padrão podem permanecer em cache na CDN por até 30 dias; URLs em private CDN/CNAME têm comportamento diferente e não são requisito do plano Free. A versão explícita da URL é a estratégia primária para evitar servir a versão anterior após substituição.
- Em substituição/remoção, desassociar a URL antiga do catálogo após confirmar a nova referência (ou imediatamente na remoção) e solicitar `destroy` com `invalidate=true`. A invalidação costuma propagar em segundos/minutos, mas cópias em navegadores, proxies e caches externos não são revogáveis pelo backend. Tratar as imagens como conteúdo público, não sensível.

3. Entrada e processamento da imagem

3.1. Formatos e limites

- Entradas aceitas: JPEG, PNG e WebP.
- Arquivos animados são rejeitados com mensagem clara; somente imagens estáticas são aceitas.

- Tamanho máximo do arquivo original: **5 MB**.
- Resolução máxima da imagem original: **40 megapixels**. Acima disso, rejeitar o arquivo.
- Validar extensão, MIME type, assinatura binária (*magic bytes*) e decodificação real. Não confiar isoladamente em extensão ou MIME enviados pelo cliente.

3.2. Pipeline obrigatório

A sequência aprovada é:

1. Receber o arquivo no backend e verificar autorização do responsável sobre o `RestaurantProduct`.
2. Validar tamanho, extensão, MIME type e assinatura binária; inspecionar cabeçalho/dimensões para rejeitar arquivos acima de 40 MP e detectar animação antes de alocar recursos de decodificação sempre que a biblioteca permitir.
3. Decodificar efetivamente a imagem; falha de decodificação interrompe o fluxo.
4. Remover metadados EXIF, incluindo geolocalização e metadados desnecessários.
5. Converter o espaço de cores para sRGB, preservando a aparência visual sempre que possível.
6. Preservar transparência quando presente no formato de entrada.
7. Redimensionar para caber em 1600×1600, preservando proporção e sem ampliar imagens menores.
8. Codificar sempre a saída em WebP, com qualidade configurável.
9. Enviar o WebP processado ao Cloudinary usando o adapter `StoragePort`, sem transformações extras na URL.
10. Persistir `public_id`, `asset_id` e `version`; somente depois da confirmação do upload, associar condicionalmente a nova `storage_key` ao produto.

A qualidade numérica do WebP, o timeout por etapa e os limites concretos de CPU/memória ainda precisam ser definidos na configuração técnica. Não se deve inventar esses valores durante a implementação.

3.3. Falha e preservação da imagem anterior

- Se validação, decodificação, processamento ou upload falhar, interromper o fluxo e registrar falha sanitizada.
- A imagem anterior permanece associada ao produto; nunca substituí-la por arquivo original não convertido ou por resultado parcial.
- O usuário pode iniciar um novo envio depois de corrigir a causa.

- Uma mesma operação de upload reutiliza sua própria `storage_key` nas tentativas; novo envio do usuário cria nova operação/chave. O adapter mantém o mapeamento estável para o `public_id` e não sobrescreve assets já existentes.

4. Persistência e reconciliação de uploads

4.1. `StorageUploadOperation`

Manter permanentemente os metadados da operação, sem que isso implique manter permanentemente o binário. Registrar, no mínimo, identificador, `RestaurantProduct`, `storage_key`, estado atual/final, timestamps e informação necessária para tentativas e reconciliações. O histórico detalhado é mantido na entidade de eventos.

Estados atuais aprovados para consulta: `PENDING`, `UPLOADED`, `ASSOCIATED`, `CLEANUP_SCHEDULED`, `FAILED` e `RECONCILING`.

4.2. `StorageUploadOperationEvent`

Entidade separada e append-only para histórico. Campos conceituais: `id`, `upload_operation_id`, `event_type`, `details` (JSONB), `actor_type`, `actor_user_id`, `process_instance_id` e `created_at`.

- `actor_type`: `USER`, `ADMIN` ou `SYSTEM`; `actor_user_id` e `process_instance_id` ficam nulos quando não aplicáveis.
- `details` usa JSONB com estrutura específica por `event_type`, com campos obrigatórios documentados; nunca guardar segredos.
- Eventos relevantes aprovados: `UPLOAD_REQUESTED`, `UPLOAD_STARTED`, `UPLOAD_COMPLETED`, `UPLOAD_ASSOCIATED`, `RECONCILIATION_STARTED`, `RECONCILIATION_RECOVERED`, `ORPHAN_DETECTED`, `CLEANUP_SCHEDULED`, `UPLOAD_FAILED`, `LEASE_CLAIMED` e `LEASE_EXPIRED`.
- Heartbeat não gera evento: atualiza somente campos operacionais da operação/lease.

4.3. Tentativas e reconciliação

- Upload síncrono pelo backend: uma tentativa inicial e até três retries para falhas transitórias, com esperas de 1 s, 3 s e 10 s.
- Repetir automaticamente apenas timeout, interrupção de conexão, HTTP 429 e HTTP 5xx. Não repetir automaticamente HTTP 400/401/403/404, salvo tratamento específico do fornecedor documentado.

- Reconciliação resolve resultado ambíguo e identifica objetos órfãos. Operações e eventos permanecem consultáveis para auditoria.
- Mensagens/códigos apresentados ao painel são estruturados e sanitizados. Preservar estágio, código seguro, status HTTP, retryability e identificador seguro de requisição do fornecedor quando disponível.
- Nunca persistir credenciais, tokens, URLs assinadas, headers de autenticação ou binários em detalhes/eventos. Stack traces ficam apenas em logs técnicos restritos e sanitizados, não no painel ADMIN.

4.4. Consulta administrativa

Endpoints aprovados, somente para consulta/investigação:

- `GET /api/admin/storage/upload-operations`
- `GET /api/admin/storage/upload-operations/{id}`

A listagem deve permitir filtrar por estado e período. Os estados disponíveis são os listados na seção 4.1. Esses endpoints não autorizam reprocessamento manual de upload quando o arquivo original não está disponível; nesse caso, o responsável deve enviar novamente após corrigir a causa.

5. Exclusão e invalidação no Cloudinary

5.1. `StorageDeletionOperation`

- Uma operação persistente controla a exclusão de um único asset, identificada por `storage_key` e `provider_asset_id`/`provider_public_id`.
- A unicidade é por `storage_key`; solicitações repetidas reutilizam o mesmo registro e não criam exclusões duplicadas.
- A operação registra tentativas, timestamps, resposta sanitizada do Cloudinary e o resultado da chamada `destroy` com `invalidate=true`. Não há ID de purge nem endpoint padrão de polling que permita reproduzir Q78/Q81 literalmente.
- Estados recomendados: `PENDING`, `DELETE_REQUESTED`, `PROVIDER_DELETE_ACCEPTED`, `FAILED` e `RECONCILIATION_REQUIRED`. `PROVIDER_DELETE_ACCEPTED` significa que o Cloudinary aceitou/concluiu a operação de origem e recebeu a solicitação de invalidação; não significa confirmação verificável de que todos os caches externos foram limpos.

5.2. Ordem e semântica da operação

1. Na substituição, primeiro confirmar o novo asset e atualizar condicionalmente a referência do produto; na remoção, retirar a referência do catálogo.
2. Chamar `destroy` pelo `asset_id` (preferido por ser imutável) com `invalidate=true`. O Cloudinary executa a exclusão do asset e solicita invalidação das cópias na CDN na mesma operação.
3. Registrar a resposta `result=ok` como aceite/resultado da operação do fornecedor, sem converter isso em `CDN_INVALIDATION_CONFIRMED`.
4. Não executar polling de 30 segundos nem criar retries separados de consulta de purge, pois o contrato padrão não expõe ID/estado de invalidação consultável.
5. A URL versionada e o `public_id` único impedem que o catálogo continue apontando para o asset antigo; a invalidação é uma camada adicional para reduzir a permanência de cópias no edge.

A documentação do Cloudinary informa propagação usual em segundos a minutos, mas assets em URLs padrão podem permanecer cacheados na CDN por até 30 dias quando não invalidados; caches de browser/proxy/terceiros ficam fora do controle do fornecedor. Portanto, esta política não é adequada para segredo, dado pessoal ou conteúdo cuja remoção exija revogação global comprovável.

5.3. Falhas e retries

- Repetir a chamada `destroy` apenas em falhas transitórias (timeout, conexão interrompida, HTTP 429 e HTTP 5xx), com idempotência por asset e registro persistente. Não repetir erros permanentes automaticamente.
- Se a resposta for ambígua (timeout após envio), consultar o asset pelo `asset_id`/Admin API antes de repetir: asset ausente indica que a exclusão na origem ocorreu; asset presente permite retry controlado. A consulta é reconciliação da existência do asset, não confirmação da propagação da invalidação.
- Falha definitiva marca a operação `FAILED` e gera alerta administrativo. O asset não deve ser removido da referência ativa antes de a substituição/remoção lógica estar consistente; em remoção solicitada pelo usuário, a referência pode ser retirada do catálogo mesmo se a exclusão remota estiver pendente.
- Q77–Q86 ficam adaptadas para o modelo Cloudinary: persistência, idempotência, retries e auditoria permanecem; a separação CDN→Object Storage, o ID de purge, polling e a garantia de confirmação real deixam de ser aplicáveis. A decisão do usuário aceita invalidação best effort e possível permanência temporária de cópias em cache.

6. Concorrência, arquivos temporários e observabilidade

- Substituições simultâneas usam versão da imagem e atualização condicional no PostgreSQL. Uma operação baseada em versão desatualizada não pode sobrescrever silenciosamente uma imagem mais recente.
- Se o upload tiver sido concluído, mas a associação condicional perder a disputa, o objeto não associado deve ser detectado como órfão e encaminhado à limpeza/reconciliação.
- Arquivos temporários ficam em diretório controlado, são removidos ao concluir ou falhar e têm limpeza periódica para temporários abandonados.
- Definir limites de duração/recursos por processamento e controlar conversões simultâneas. Não permitir concorrência ilimitada.
- Métricas do pipeline: sucesso, falha, duração, retries, volume processado e consumo de recursos, com possibilidade de identificar operações problemáticas.

7. Critérios de aceite e testes mínimos

1. Rejeitar extensão/MIME/assinatura incompatíveis e arquivos que não decodificam.
2. Rejeitar original acima de 5 MB, acima de 40 MP ou animado.
3. Confirmar entrada JPEG/PNG/WebP e saída sempre WebP.
4. Verificar EXIF removido, sRGB, transparência preservada, proporção mantida e ausência de upscale.
5. Confirmar limite máximo de 1600×1600 e qualidade configurável.
6. Simular falha em cada estágio e verificar que a imagem anterior permanece associada.
7. Simular timeout, conexão interrompida, 429, 5xx e erros permanentes; verificar política de retry e intervalos.
8. Reiniciar processo durante upload/reconciliação e verificar recuperação a partir de registros persistidos.
9. Verificar eventos append-only e sanitização: nenhum segredo, URL assinada, header de autenticação ou binário.
10. Simular substituições concorrentes; garantir compare-and-swap, ausência de sobrescrita obsoleta e limpeza de órfãos.
11. Verificar `destroy` por asset ID com `invalidate=true`, resposta sanitizada, retries transitórios e reconciliação de resultado ambíguo; não simular confirmação de purge nem polling inexistente.
12. Repetir pedido de exclusão para o mesmo `storage_key` e verificar reutilização da operação única.

13. Verificar `CLOUDINARY_CLOUD_NAME` e secrets separados por ambiente, URLs versionadas e ausência de exposição do `api_secret`.
14. Verificar limites de tempo/recursos, concorrência limitada, limpeza de temporários e métricas do pipeline.

8. Pendências que não bloqueiam a definição conceitual, mas precisam ser fechadas antes da implementação afetada

- Implementar/configurar a conta Cloudinary Free sem cartão e validar os limites efetivos do ambiente; nenhuma contratação ou upgrade é permitido sem autorização explícita do proprietário.
- Definir valores concretos de timeout, CPU, memória e limite de conversões simultâneas.
- Definir qualidade numérica WebP e configuração por ambiente.
- Fechar schema físico, enums, índices e política de retenção/consulta das operações.
- Definir DTO/contrato da operação de upload do responsável e a resposta pública com URL versionada do Cloudinary; não foi aprovado um novo path HTTP de upload nesta entrevista.
- Definir o mecanismo/endpoint de reprocessamento administrativo de `StorageDeletionOperation`; os endpoints de upload aprovados são somente de consulta.
- Documentar campos obrigatórios JSONB por tipo de evento e os códigos de erro sanitizados do Cloudinary.
- Implementar monitoramento de consumo pela Admin API Usage do Cloudinary. A decisão de produto é bloquear novos uploads ao atingir o limite gratuito e preservar as imagens existentes; como salvaguarda técnica, recomenda-se configurar bloqueio preventivo ao atingir 90% da estimativa de créditos, com alerta ADMIN. **O patamar de 90% é uma margem operacional recomendada/configurável, não uma decisão de produto independente já aprovada.**
- A Admin API Usage fornece dados atualizados periodicamente, não uma leitura em tempo real. A implementação deve cachear resultados e consultar em intervalo conservador, respeitando os limites de chamadas da conta/Plano Free. Se os dados estiverem atrasados ou indisponíveis, não liberar uploads ilimitados: manter o último valor válido com idade registrada e aplicar o bloqueio conservador quando não for possível assegurar a margem gratuita.
- O bloqueio não interrompe entrega das imagens existentes, que continua consumindo banda; não há fallback automático, upgrade ou cobrança. Os limites do plano são soft limits do fornecedor; o bloqueio interno reduz risco, mas não garante que

Cloudinary interrompa serviço/consumo exatamente no mesmo instante.

- Validar em conta de desenvolvimento o comportamento de `destroy` com `invalidate=true`, limites da API e reconciliação de exclusões ambíguas.

9. Matriz de rastreabilidade Q63–Q103

Questão	Decisão aprovada
Q63	Manter permanentemente os metadados da operação de upload e seus dados de reconciliação/tentativas; isso não exige reter o binário.
Q64	Histórico em entidade `StorageUploadOperationEvent` separada e append-only.
Q65	Registrar `actor_type` (USER/ADMIN/SYSTEM), `actor_user_id` e `process_instance_id`, nulos quando não aplicáveis.
Q66	Usar JSONB em `details`, com estrutura/campos obrigatórios por tipo de evento e sem segredos.
Q67	Registrar os eventos relevantes definidos na seção 4.2; heartbeat atualiza campos operacionais sem criar evento.
Q68	Upload síncrono pelo backend, uma tentativa inicial + até três retries em 1 s, 3 s e 10 s; falha final preserva a imagem anterior.
Q69	Retry automático somente para timeout, conexão interrompida, HTTP 429 e HTTP 5xx; erros permanentes não são repetidos automaticamente.
Q70	Endpoints ADMIN de consulta: lista e detalhe de `StorageUploadOperation`; sem reprocessamento manual de upload sem arquivo original.

Questão	Decisão aprovada
Q71	Listagem administrativa com filtros por estado e período; estados PENDING, UPLOADED, ASSOCIATED, CLEANUP_SCHEDULED, FAILED e RECONCILING.
Q72	Erros estruturados e sanitizados; não expor credenciais, tokens, URLs assinadas, headers de autenticação, binários ou stack traces no painel.
Q73 (adaptada)	`storage_key` lógica única por operação, mapeada no adapter para `public_id` imutável; substituição cria novo asset e remoção usa `StorageDeletionOperation`.
Q74	Cloudinary escolhido para o MVP; backend exclusivo para upload/substituição/exclusão; entrega pública por CDN gerenciada pelo Cloudinary.
Q75	URLs imutáveis/únicas e invalidação da URL antiga quando a imagem for substituída/removida.
Q76 (adaptada)	Fluxo persistente `StorageDeletionOperation` coordena exclusão do asset Cloudinary e solicitação de invalidação best effort.
Q77 (adaptada)	`destroy` do Cloudinary com `invalidate=true` executa exclusão e solicita invalidação na mesma chamada; não há etapa independente de confirmação de purge.
Q78 (adaptada)	Registrar a resposta do `destroy` como resultado do fornecedor; não declarar confirmação de propagação global do cache, pois não há ID/status padrão consultável.

Questão	Decisão aprovada
Q79 (adaptada)	Manter retries/backoff para falhas transitórias da chamada `destroy`; não há retries independentes de CDN e origem.
Q80 (adaptada)	Uma `StorageDeletionOperation` controla a chamada única de exclusão/invalidação do Cloudinary e sua reconciliação.
Q81 (substituída)	Não aplicável: o Cloudinary não expõe status de purge consultável no contrato padrão; reconciliar existência do asset quando a resposta for ambígua.
Q82 (adaptada)	Repetir `destroy` apenas para falhas transitórias; não repetir erros permanentes.
Q83	Unicidade da `StorageDeletionOperation` por `storage_key`; pedidos repetidos reutilizam a operação.
Q84 (adaptada)	Falha definitiva do `destroy` marca operação FAILED e alerta ADMIN; não há garantia de preservar cópia em cache ou confirmar purge.
Q85 (substituída)	Não aplicável como consulta de purge; usar reconciliação da existência do asset em resultados ambíguos.
Q86 (adaptada)	Usar public_id e URLs versionadas imutáveis; não prometer cache de um ano no CDN padrão. O cache pode permanecer por até 30 dias e invalidação é best effort.
Q87 (adaptada)	PostgreSQL mantém chave lógica e metadados necessários do asset Cloudinary (`asset_id`, `public_id`, `version`); adapter gera URL versionada.

Questão	Decisão aprovada
Q88 (adaptada)	Configurar `CLOUDINARY_CLOUD_NAME` por ambiente; usar URL versionada retornada pelo adapter, sem pressupor domínio CDN próprio.
Q89	Aceitar JPEG, PNG e WebP como entrada e converter sempre para WebP.
Q90	Limite de 5 MB para o arquivo original.
Q91	Redimensionar até 1600×1600, preservando proporção, sem ampliar imagens menores; qualidade WebP configurável.
Q92	Remover metadados EXIF durante o processamento.
Q93	Preservar transparência quando presente no formato de entrada.
Q94	Aceitar somente imagens estáticas; rejeitar animações com mensagem clara.
Q95	Falha em decodificação/processamento interrompe upload, registra falha e preserva imagem anterior.
Q96	Validar extensão, MIME type, magic bytes e decodificação real.
Q97	Rejeitar imagem original acima de 40 megapixels.
Q98	Converter para sRGB, preservando a aparência visual sempre que possível.
Q99	Limitar duração/recursos e conversões simultâneas; registrar falhas por esgotamento de recursos.
Q100	Seguir a ordem de pipeline especificada na seção 3.2.
Q101	Usar diretório temporário controlado, limpar no sucesso/falha e remover abandonados periodicamente.

Questão	Decisão aprovada
Q102	Usar versão da imagem e atualização condicional para impedir sobrescrita obsoleta em concorrência.
Q103	Medir sucesso, falha, duração, retries, volume processado e consumo de recursos.

10. Referências oficiais do fornecedor

- Cloudinary — Billing and plans: plano Free, créditos, ausência de cartão no cadastro e comportamento de excedentes.
- Cloudinary — Invalidate cached assets: invalidação, propagação e comportamento de cache por tipo de entrega.
- Cloudinary — Upload API reference: `destroy`, `asset_id`, `invalidate` e respostas da API.
- Cloudinary — Admin API: consulta de recursos e operações administrativas, incluindo uso/limites.

Os limites e condições do plano são definidos pelo fornecedor e podem mudar. Validar a conta e os limites efetivos no momento da implantação; esta documentação não substitui a tela de billing da conta Cloudinary. O endpoint de uso não deve ser tratado como métrica em tempo real; registrar timestamp/idade de cada leitura e não exceder rate limit da Admin API.